Async credit processing service

Architecture and key decisions

Accept credits durably and apply them asynchronously. **Each credit changes the balance at most once**, even when requests or tasks repeat. MongoDB owns balances and operation state; RabbitMQ transports work; Celery executes it.

Request flow

1. **Validate and accept.** FastAPI validates `POST /credits`. One MongoDB transaction creates the credit, increments the user's pending count, and sets status to `processing`.
2. **Publish and respond.** The API attempts publication, then returns **202 Accepted**, a stable `task_id`, and a `Location`. It does not wait for worker completion.
3. **Recover dispatch.** A separate dispatcher claims expiring leases on pending credits and republishes work after failed or interrupted publication.
4. **Apply and observe.** The worker atomically increments `balance`, updates counters/status, and marks the credit `processed`. GET endpoints expose individual outcomes and user balances.

Data model and design rationale

- **Two collections.** Users hold balances and pending/failed counters. Credits hold request identity, amount, status, timestamps, dispatch lease, and error. No separate Celery result backend is needed.
- **Database constraints.** Unique user IDs and idempotency keys prevent races. A status/dispatch-time index supports pending-work scans.
- **Exact money values.** Python `Decimal` and BSON `Decimal128` preserve decimal arithmetic. Responses use decimal strings; `$inc` creates a missing balance field.
- **Atomic completion.** Snapshot reads and majority writes couple money with the processed marker. Retriable transaction callbacks contain only database work. Worker clients are created after Celery forks.

API assumptions and status semantics

- **Amounts:** one fixed currency; positive, finite, at most two decimal places; maximum one billion major units. Unknown users are created.
- **Request retries:** `Idempotency-Key` returns the original credit. Changed data with the same key returns **409**; invalid input returns **422**. Without a key, each call creates a new credit.
- **User status:** `processing` while credits are pending; otherwise `failed` if unresolved failures remain; otherwise `processed`. Each credit retains its own outcome.

Reliability testing and delivery

Failure handling and recovery

- **Publication gap.** Pending credits double as durable dispatch records. The API publishes initially; the dispatcher republishes after a **30-second lease** expires, recovering interrupted publication.
- **Duplicate delivery.** Persistent messages, publisher confirms, and late acknowledgments support recovery. The completion transaction prevents a second increment after acknowledgment is lost.
- **Transient failures.** Connections retry with jitter and capped exponential delay; transient transactions retry. An infrastructure outage alone does not permanently fail a credit.
- **Permanent failures.** Non-transient database errors become `failed` when persistable; unexpected errors remain pending and logged. Operators fix and requeue failures. Processed credits cannot be reopened.

Testing strategy and evidence

- **Validation and API:** amount precision/limits, user IDs, acceptance, polling, conflicts, missing records, database unavailability, and failed publication.
- **Transaction correctness:** real MongoDB tests cover concurrency, duplicate execution, aggregate status, failed-credit recovery, and rollback after an injected mid-transaction exception.
- **Complete workflow:** concurrent requests cross real RabbitMQ, Celery, and MongoDB. **All 34 tests passed**, plus real worker/broker recovery checks. Post-commit redelivery was simulated; OS-level crash testing remains outstanding.

Local operation

- **Reproducible setup:** Docker Compose, a transaction-capable MongoDB replica set, health checks, persistent volumes, non-root application containers, and locked Python dependencies.
- **Visible recovery:** logs use task IDs; the CLI reports pending/failed counts and oldest pending age. MongoDB determines API readiness. The dispatcher recovers broker outages once dependencies return.

Trade-offs and production boundaries

- **Queue traffic:** periodic republication can duplicate work already queued. At scale, add admission control, adaptive dispatch, queue-age alerts, and a refined outbox/reconciliation policy.
- **Contention and latency:** transactions contend on busy accounts; driver retries can extend request time. Hard HTTP deadlines and measured capacity targets need additional design.
- **Launch gates:** account authorization, tenant-scoped keys, TLS/secrets, replicated infrastructure, tested restores, reconciliation, SLOs, and failover drills. Local infrastructure is not highly available; payment rails, UI, and cloud deployment are outside scope.